

How TIDE works

A plain-English and technical walkthrough

Cache replacement project

September 27, 2026

Contents

1	The problem in one paragraph	1
2	Baseline: what ARC does on its own	1
3	What TIDE adds: a learned victim	2
3.1	Exactly which candidates are scored	3
4	The features: what the network is shown	3
5	The network and how it decides	4
5.1	Turning scores into a decision	4
6	Learning while the cache runs	4
6.1	The exact training step	5
6.2	Cold start: no guessing before there is evidence	5
7	The safety net: never much worse than ARC	6
8	Putting it together: what happens on one request	6
9	A worked mini-example	6
10	Why not PyTorch or TensorFlow	7
11	Headline results	7

1 The problem in one paragraph

A cache is a small, fast storage area that holds a limited number of items so that repeat requests for them are served quickly instead of going to slow storage. When the cache is full and a new item arrives, something already in the cache has to be thrown out to make room. Which item to throw out is the *eviction* decision, and it is the only thing that separates a good cache policy from a bad one: every policy stores the same items, they just disagree about which one to evict. TIDE is a new eviction policy. It wraps a well-known policy called ARC and adds a small, constantly learning model that makes the eviction choice ARC would otherwise make on its own. This document explains, in order: what ARC does, why it is not enough on its own, what TIDE adds, exactly what numbers it computes for each candidate, the exact network and how it trains itself while the cache is running, and how it protects itself from its own mistakes. Every rule stated here is the rule implemented in `tide.py` and `policies.py`; there is no simplification that changes behaviour.

2 Baseline: what ARC does on its own

ARC (*Adaptive Replacement Cache*, Megiddo & Modha, 2003) keeps four lists instead of the one list a simple cache uses:

- **T1**: items currently in the cache that have been requested *once recently*.

- **T2**: items currently in the cache that have been requested *at least twice*, i.e. they have shown up again since they were last considered "new".
- **B1**: a *ghost list* — not actual data, just a memory of keys recently evicted from T1.
- **B2**: a ghost list of keys recently evicted from T2.

The point of B1 and B2 is that ARC can notice when it evicted something too early. If a key that was in B1 comes back, that is evidence ARC is throwing away "recency" items too fast, so it grows T1 at T2's expense. If a key from B2 comes back, ARC was throwing away "frequency" items too fast, so it shrinks T1. This growing and shrinking is controlled by a single number, p , which is the target size of T1.

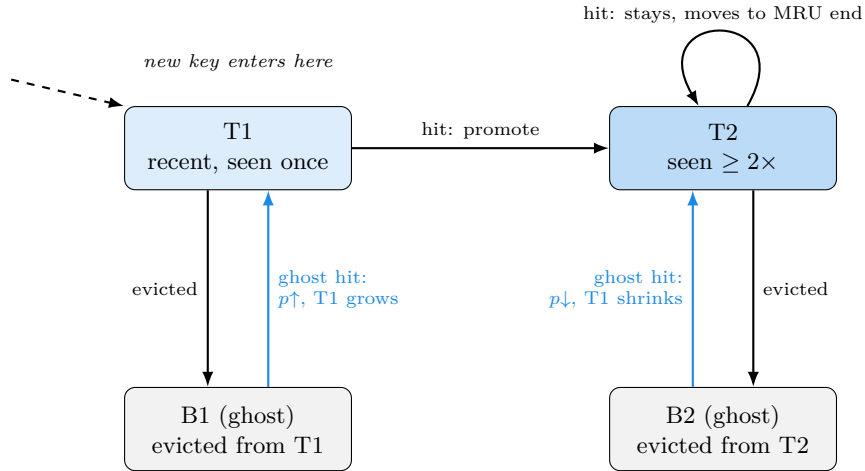


Figure 1: ARC's four lists. T1 and T2 hold the actual cached items; B1 and B2 remember only the *keys* of recently evicted items, so ARC can tell whether an eviction was a mistake.

Formally, on a ghost hit:

$$\text{B1 hit: } p \leftarrow \min\left(c, p + \max\left(1, \frac{|B2|}{|B1|}\right)\right) \quad \text{B2 hit: } p \leftarrow \max\left(0, p - \max\left(1, \frac{|B1|}{|B2|}\right)\right)$$

where c is the cache capacity. When something must be evicted, ARC's **REPLACE** rule decides *which list* loses an item: it evicts from T1 if T1 is bigger than its target p (or a tie-break case), otherwise from T2. Whichever list is chosen, plain ARC always evicts that list's *least-recently-used* (LRU) item — the one nobody has asked for in the longest time.

Where ARC falls short. ARC decides *which list* to shrink well, using real feedback, but inside that list it falls back to plain "oldest first", which cannot see repeating patterns. Two examples that break it, both measured on real traces during this project:

- **A loop bigger than the cache.** If items $1, 2, \dots, 17$ are requested over and over with a cache of size 16, ARC (and LRU) get a **0% hit rate** — the item ARC always evicts is always the very next one requested. TIDE gets that same trace above 40% higher than ARC and LRU (see `test_tide.py::test_learns_loop`).
- **A one-time scan.** A burst of never-repeated keys pushes every genuinely useful item out of T1's LRU end before ARC even realises the burst is over.

3 What TIDE adds: a learned victim

TIDE keeps ARC's four lists and its p update rule completely unchanged. The only thing that changes is *which item* is evicted from the list ARC picked. Instead of always taking the oldest item, TIDE scores a handful of candidates with a small neural network and evicts whichever one the network thinks is safest to lose — i.e. the one that will not be needed again for the longest time.

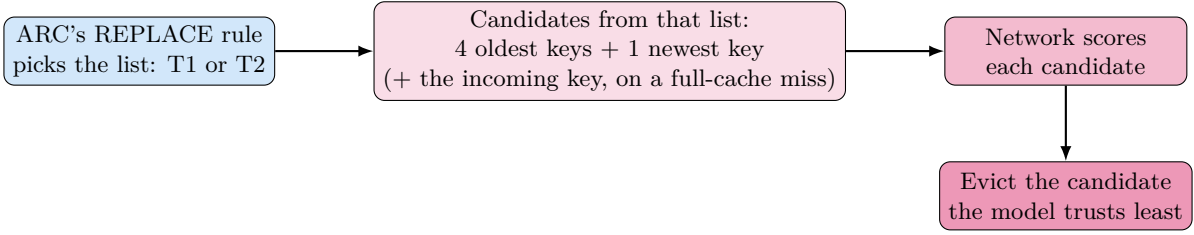


Figure 2: TIDE’s decision pipeline for one eviction. ARC still decides the list; TIDE only decides the victim within it.

3.1 Exactly which candidates are scored

For the list L that ARC’s rule selected, TIDE builds a candidate set of up to six keys (`tide.py`, method `_choose`):

1. The **4 least-recently-used** keys of L (its "cold" end) — this is what ARC alone would pick from, and is always candidate 0.
2. The **most-recently-used** key of L , if L has more than 4 items. This sounds counter-intuitive — why would you ever evict the item you just used? — but it is essential for long, repeating cycles: if a group of keys will not be needed again for a very long time (a full cycle), the smartest move can be to evict the one you *just* finished with, since it has the most time before it is needed again.
3. On a **miss with a full cache**, the **incoming key itself**. If the model chooses this one, the new request is simply not cached at all. This is called **bypass**, and it is the only way to survive a one-time scan or a loop bigger than the cache: caching a key you will never (or not for a very long time) see again just evicts something more useful in its place.

4 The features: what the network is shown

For every candidate key k , TIDE keeps a small history record and turns it into 12 numbers before handing it to the network. The record for each key, kept in a dictionary keyed by the item id (`tide.py`, method `_touch`), is:

$$[t_{\text{last}}, g_1, g_2, g_3, \text{ewma}, C_1, C_2, C_3, \text{count}]$$

where t_{last} is the time of the last access, g_1, g_2, g_3 are the last three gaps between consecutive requests for that key (most recent first), ewma is an exponentially smoothed average gap ($\text{ewma} \leftarrow 0.7 \cdot \text{ewma} + 0.3 \cdot \text{gap}$), C_1, C_2, C_3 are three counters that decay at different speeds (half-lives of c , $4c$ and $16c$ requests, so C_1 reacts fast and C_3 remembers long-term popularity), and count is the total number of times the key has ever been seen. This record survives eviction: it is kept for up to $8c$ keys (whether they are currently cached or not), so a key that left the cache and comes back later is recognised, not treated as brand new.

At the moment a key is scored, its 9-number record is converted into the 12 features below (`tide.py`, method `_feat`). Every time-like quantity is passed through $\lg(v) = \log(1 + v/c)$: dividing by the cache size c makes the features comparable across different cache sizes, and $\log(1 + \cdot)$ keeps a request that is 10 apart and one that is 10,000 apart from swamping each other.

#	Feature	Meaning
1	$\lg(\text{age})$	how long since this key was last requested
2–4	$\lg(g_1), \lg(g_2), \lg(g_3)$	the last three gaps between requests for this key (a missing gap uses a fixed "never seen" value, $\log(1 + 2W/c)$)
5	$\lg(\text{ewma})$	smoothed recent gap length
6	$\min(\text{count} - 1, 3)/3$	how many times seen, capped and scaled to $[0, 1]$
7–9	three decayed counts	popularity at three different memory lengths (fast/medium/slow forgetting)
10	$\log(1 + \text{count})$	total times ever seen
11	<code>is_incoming</code>	1 if this candidate is the new key trying to get in, else 0
12	<code>in_T2</code>	1 if this candidate currently sits in ARC’s "frequent" list, else 0

Table 1: The 12 input features per candidate. Nothing here needs the item’s actual content — only its request history — so it works for opaque numeric keys exactly as well as for URLs or filenames.

There are deliberately no hand-built "trend" features (such as "is this key rising or falling in popularity"). A rising key already looks different from a falling key through features 1–10 alone (short recent gaps and a high fast counter vs. long gaps and a fading fast counter), and a small neural network combines raw features into exactly this kind of pattern on its own during training.

5 The network and how it decides

The network is a plain two-layer network with one hidden layer, written from scratch in NumPy (no PyTorch or TensorFlow is needed for something this small — see the discussion below):

$$\hat{y} = w_2^\top \max(0, W_1^\top x + b_1) + b_2, \quad x \in \mathbb{R}^{12}, W_1 \in \mathbb{R}^{12 \times 64}, w_2 \in \mathbb{R}^{64}$$

It has one output, $\hat{y}$, which is trained to predict $\log(1 + d/c)$, where d is *how many requests from now this key will next be asked for*. A large $\hat{y}$ means "not needed again for a long time" — exactly the item you want to evict.

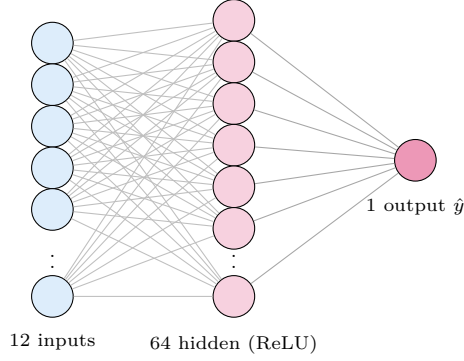


Figure 3: The network: 12 features $\rightarrow$ 64 hidden units with a ReLU ("keep positive values, zero out negative ones") $\rightarrow$ one predicted reuse-time.

5.1 Turning scores into a decision

Given the scores $\hat{y}_0, \dots, \hat{y}_n$ for the $n+1$ candidates (index 0 is always ARC's own choice, the oldest key), TIDE does *not* blindly take the maximum. It evicts

$$\arg \max_i (\hat{y}_i - m_i), \quad m_i = \begin{cases} 0 & i = 0 \text{ (ARC's own candidate)} \\ \delta = 0.1 & i \in \{1, 2, 3\} \text{ (other tail candidates)} \\ \delta_{\text{far}} = 0.2 & i \text{ is the MRU key or the incoming key} \end{cases}$$

The margins δ and δ_{far} mean the model has to be *clearly* more confident than ARC's own guess before it is allowed to override it, and needs to be even more confident to make the riskier MRU or bypass choice. These numbers were picked once, by testing a grid of settings across every trace used in the benchmark, and then fixed for all traces — nothing here is tuned per trace.

6 Learning while the cache runs

This is the part that lets TIDE improve over time instead of being trained once and frozen. The key idea is: *every candidate the model ever scores eventually gets a correct answer for free*, because the cache keeps running and either sees that key again or does not.

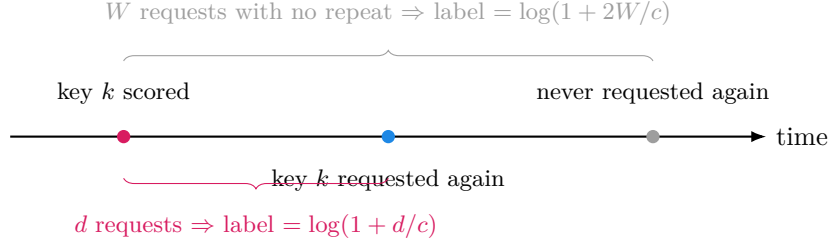


Figure 4: Every scored candidate resolves into a training example, one way or the other, with no need for a human or a special "ground truth" file. The two outcomes (reused after d requests, or censored after W) are alternatives for the same candidate, not two separate events.

Concretely (`tide.py`, methods `_choose`, `access`, `_label`):

1. Whenever a candidate is scored (whether or not the model is even switched on yet, see §6.2), its 12 features are copied into a *pending* table, timestamped with the current request number t_0 .
2. On every later request for key k : if k has a pending entry, it is resolved with label $\log(1 + d/c)$ where $d = t - t_0$ is exactly how many requests it took to come back. This happens *even on a cache hit* — resolving labels does not depend on whether the request was a hit or a miss.
3. If $W = 8c$ requests pass with no repeat, the pending entry expires and is resolved as *censored*: labelled $\log(1 + 2W/c)$, the LRB convention of "twice the window", so the model learns "very far away" rather than a wrong precise number.
4. Each resolved (features, label) pair is written into a 32,768-row *replay buffer* that overwrites its oldest rows once full, so old, possibly outdated patterns are gradually forgotten and never pinned forever.
5. Every 32 new labels, one training step runs: sample 256 random rows from the replay buffer and take one Adam optimiser step to reduce the mean-squared error between the network's prediction and the true label.

6.1 The exact training step

Given a batch of 256 (features, label) pairs (x_i, y_i) , the loss is mean squared error, $\mathcal{L} = \frac{1}{256} \sum_i (\hat{y}_i - y_i)^2$, and every weight is updated with Adam (Kingma & Ba, 2015), which keeps a running average of the gradient (m) and of its square (v) so that each of the roughly 1,600 numbers in the network gets its own, self-adjusting step size:

$$m \leftarrow 0.9m + 0.1g \quad v \leftarrow 0.999v + 0.001g^2 \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}} + \varepsilon} \hat{m}$$

with $\hat{m}, \hat{v}$ the usual bias-corrected versions and learning rate $\eta = 10^{-3}$. All four weight tensors (W_1, b_1, w_2, b_2 , about 1,601 numbers for the default 64 hidden units) are stored in one flat array so the whole update above is a handful of NumPy vector operations rather than a loop.

6.2 Cold start: no guessing before there is evidence

TIDE does not turn its model on immediately. It keeps collecting labels and training in the background, but every eviction is decided exactly as plain ARC would decide it, until *all* of these are true:

- at least $W = 8c$ requests have happened (so the "never came back" signal is meaningful), and
- at least 2,048 labels have been collected (so the replay buffer isn't mostly empty), and
- the model is not currently in a fallback cool-down (next section).

This means the very first stretch of any trace is identical to plain ARC, and TIDE only starts overriding decisions once it has real evidence to act on.

7 The safety net: never much worse than ARC

A model can be wrong, especially right after the workload changes. TIDE protects against this with a second, independent copy of plain ARC running silently alongside it (the *shadow*), fed the exact same requests but never influenced by TIDE's decisions.

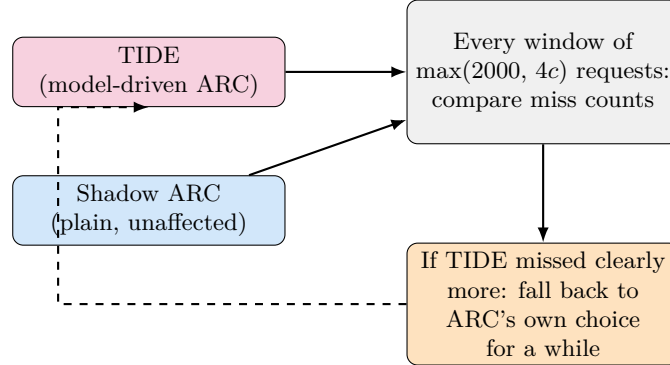


Figure 5: The shadow comparison and fallback loop. The fallback path is dashed because it feeds back into future decisions, not the current one.

Every $\max(2000, 4c)$ requests, TIDE compares its own miss count to the shadow's. If TIDE missed more by more than $\max(10, 1\% \text{ of the window})$, it falls back to letting ARC's own rule choose the victim for a "cool-down" period. If this keeps happening again and again, the cool-down doubles each time (2, 4, 8, 16 windows, capped), so a workload that keeps confusing the model gets progressively more of ARC's steady behaviour instead of the model's guesses. If TIDE goes a window without missing more than the shadow, the failure count relaxes back down, so a temporary rough patch does not permanently poison the policy. Two details worth being explicit about, since they are easy to get backwards:

- The shadow is *only* used to detect regime changes and decide on fallback. It is never used as a source of "regime" features for the model itself, because if it were, the model's own past mistakes (which count against the real cache, not the shadow) would leak into its inputs.
- Label collection and training *continue during fallback*. The model keeps learning even while it isn't making decisions, so once the workload settles back down it can immediately re-activate with a model that has kept up to date.

8 Putting it together: what happens on one request

1. Advance the clock. Expire any pending candidate snapshot older than W requests (censored label). If the requested key has a pending snapshot, resolve it with the true gap d .
2. Update that key's history record (last three gaps, smoothed average, three decayed counters, total count).
3. Feed the request to the shadow ARC (for later comparison only).
4. Run ARC's normal logic: hit in T1 or T2, or a ghost hit in B1/B2 (which updates p), or a genuine miss.
5. If a miss requires an eviction: ARC's rule (unchanged) decides the list; TIDE scores up to 6 candidates from that list (Section 3) and, if the model is switched on, evicts the one with the lowest trusted score; otherwise it falls back to ARC's own oldest-item choice.
6. Every 32 newly resolved labels, run one training step (Section 5.1).
7. Every $\max(2000, 4c)$ requests, compare against the shadow and update the fallback state (Section 6).

9 A worked mini-example

Suppose the cache holds keys $\{5, 9, 12, 20\}$ in T1's cold-to-hot order, cache size $c = 4$, and key 20 is the most-recently-used key of T1. A miss now forces an eviction from T1. The four tail candidates are 5, 9, 12, 20 (note: 20 also happens to be the MRU key here since T1 has only 4 items, so no separate MRU candidate is added).

Suppose their fast decayed counters and last-gap history give the model these predicted values (already net of the margin, for illustration):

$$\hat{y}_5 = 0.9 \quad (\text{not requested in ages, low frequency}) \quad \hat{y}_9 = 2.4 \quad (\text{part of a big loop, due back in a long time})$$

$$\hat{y}_{12} = 0.3 \quad (\text{steadily popular, likely requested very soon}) \quad \hat{y}_{20} = 1.1 \quad (\text{ARC's own pick, the plain LRU choice})$$

With margins $\delta = 0.1$ applied to candidates 9, 12, 20 (all "other tail" candidates) and none to the ARC default (index 0, key 5 in this ordering): 5: 0.90, 9: 2.30, 12: 0.20, 20: 1.00. The highest adjusted score is key 9's, so TIDE evicts key 9, not key 5 (ARC's own pick) and not the still-frequently-needed key 12. Some time later, whichever of these keys is next requested resolves that key's label with the true gap, and the network is nudged toward whichever prediction turns out to have been right.

10 Why not PyTorch or TensorFlow

The network here has about 1,600 numbers and is called on 4–6 rows at a time, thousands of times per second, inside a single-threaded cache simulator. A deep-learning framework's per-call overhead (tens of microseconds just to enter and leave the framework) would dominate the actual arithmetic, which NumPy does directly with essentially no overhead. Bringing in a roughly gigabyte dependency to save about 20 lines of hand-written gradient and Adam code (which is fully tested, see `test_tide.py::test_grad`, a numerical gradient check) is not worth it here. It would be worth it if the network grew much larger, or for an *offline* teacher model such as a small transformer trained once on a GPU and then distilled into a model this size for the actual cache — an idea discussed as "Option C" in the project's design document, `REPLAN.md`.

11 Headline results

Full detail, every trace and cache size, is in the companion document `report.pdf`. In short, on a 43-cell benchmark (12 real/synthetic traces $\times$ 3 cache sizes, plus 3 generated 1-million-request trend traces $\times$ 3 cache sizes), averaged over 3 random seeds:

- TIDE has the best mean normalized hit rate of the compared policies: 0.379 vs. 0.306 for ARC and 0.334 for S3-FIFO (0 = no better than the simplest policy, LRU; 1 = as good as the theoretical optimum, Belady's algorithm, which requires knowing the future).
- TIDE has the single highest hit rate in 24 of 43 cells, and is at least as good as plain ARC in 34 of them.
- On the loop-shaped "adversarial" trace, TIDE turns a **0%** hit rate into roughly **+48.6** percentage points, purely through the bypass mechanism in Section 3.
- A decision takes about 15 μ s at the median and never took more than 30 μ s at the 99th percentile in these benchmarks; a cache *hit* never touches the model at all.